

Anno Accademico 2025/2026

Leonardo Donati

Appunti  
**Ingegneria del Software**

*Teoria e esercizi.*



**UNIMORE**  
UNIVERSITÀ DEGLI STUDI DI  
MODENA E REGGIO EMILIA

# Indice

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Scrivere un programma</b>                                  | <b>4</b>  |
| <b>2</b>  | <b>Costruire un programma</b>                                 | <b>5</b>  |
| 2.1       | Dimensione e Complessità . . . . .                            | 5         |
| 2.1.1     | Divide et Impera . . . . .                                    | 5         |
| 2.2       | CI/CD (Continuous Integration/Continuous Deployment . . . . . | 5         |
| <b>3</b>  | <b>Ingegneria del Software</b>                                | <b>6</b>  |
| 3.1       | Processi Software . . . . .                                   | 7         |
| <b>4</b>  | <b>Ingegneria dei Requisiti</b>                               | <b>8</b>  |
| 4.1       | Elicitation Process . . . . .                                 | 8         |
| 4.2       | Analisi dei Requisiti . . . . .                               | 9         |
| 4.3       | Definizione, Prototipazione e Review . . . . .                | 9         |
| 4.4       | Requirement specification . . . . .                           | 9         |
| <b>5</b>  | <b>Design SOLID</b>                                           | <b>10</b> |
| 5.1       | Architettura e Design . . . . .                               | 10        |
| 5.2       | Principi SOLID . . . . .                                      | 10        |
| 5.3       | Principi di coesione dei componenti . . . . .                 | 11        |
| 5.4       | Principi di accoppiamento dei componenti . . . . .            | 11        |
| <b>6</b>  | <b>Architettura del software</b>                              | <b>12</b> |
| 6.1       | Elementi dell'architettura del software . . . . .             | 12        |
| 6.1.1     | Caratteristiche Architetturali . . . . .                      | 12        |
| 6.1.2     | Decisioni Architetturali . . . . .                            | 13        |
| 6.1.3     | Componenti Logici . . . . .                                   | 13        |
| 6.1.4     | Stili Architetturali . . . . .                                | 13        |
| <b>7</b>  | <b>Architettura Pulita</b>                                    | <b>16</b> |
| 7.1       | Livelli di Disaccoppiamento . . . . .                         | 16        |
| 7.2       | Architettura Pulita . . . . .                                 | 17        |
| 7.3       | Documentazione 4+1 . . . . .                                  | 17        |
| 7.4       | Modello C4 . . . . .                                          | 17        |
| <b>8</b>  | <b>Metriche di Qualità</b>                                    | <b>18</b> |
| 8.1       | Metrica Software . . . . .                                    | 18        |
| 8.1.1     | Misure Statiche . . . . .                                     | 18        |
| 8.1.2     | Metriche Dinamiche . . . . .                                  | 20        |
| <b>9</b>  | <b>Modelli e UML</b>                                          | <b>21</b> |
| 9.1       | UML . . . . .                                                 | 21        |
| <b>10</b> | <b>Implementazione</b>                                        | <b>22</b> |
| 10.1      | Modello di Qualità del Prodotto Software . . . . .            | 22        |
| 10.2      | Modello di Qualità d'Uso . . . . .                            | 22        |
| 10.3      | Debugging . . . . .                                           | 23        |
| 10.4      | Assertion and Defensive Programming . . . . .                 | 23        |
| 10.5      | Ottimizzazione delle Performance . . . . .                    | 23        |
| 10.6      | Software Manutenibile . . . . .                               | 23        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>11 Testing</b>                                     | <b>25</b> |
| 11.1 Software Development Life Cycle (SDLC) . . . . . | 25        |
| 11.2 Componenti del Testing . . . . .                 | 26        |
| 11.3 Processo di Testing . . . . .                    | 27        |
| 11.4 Processo di revisione . . . . .                  | 27        |
| 11.5 Black Box Testing . . . . .                      | 28        |
| 11.6 White Box Testing . . . . .                      | 28        |
| <b>12 Design Pattern</b>                              | <b>30</b> |
| 12.1 Strategy Pattern . . . . .                       | 31        |
| 12.2 Observer Pattern . . . . .                       | 32        |
| 12.3 Decorator Pattern . . . . .                      | 33        |
| 12.4 Factory Pattern . . . . .                        | 34        |
| 12.5 Singleton Pattern . . . . .                      | 35        |

# Capitolo 1

## Scrivere un programma

Per scrivere correttamente un programma ci sono diverse fasi importanti:

- **Specifica dei requisiti:**

1. **Parte teorica:** obiettivi, input attesi, modalità per generare le specifiche.
2. **Parte applicativa:** uso dello **use-case diagram UML** per la rappresentazione dei requisiti.
3. **Requisiti funzionali:** richieste del programma, funzionalità.
4. **Specifiche non funzionali:** vincoli di design:
  - (a) **Requisiti di performance:** tempo massimo in base alla dimensione degli input.
  - (b) **Requisiti di tempo-reale:** completare il processing in un tempo utile all'utente.
  - (c) **Requisiti di modificabilità/manutenibilità:** aspettative di vita del programma e se andrà modificato.
  - (d) **Interfaccia utente:** CLI oppure GUI.
  - (e) **Dimensioni massime dell'input:** così scelgo algoritmi che ottimizzano le performance.
  - (f) **Piattaforma di sviluppo:** si intende architettura del sistema, OS, librerie da usare per la portabilità.
  - (g) **Requisiti di tempo/scadenza:** ovviamente la deadline impatta i costi.
  - (h) **Linguaggio di programmazione:** di solito è una scelta di design ma potrebbe essere fissata.
  - (i) **Algoritmi:** anche questi possono essere imposti, ma di solito le performance influenzano la scelta.
  - (j) **Struttura del software.**

- **Progettazione:**

1. **Parte teorica:** stili architetturali, decomposizione funzionale, ORM, definizione e problematiche, metriche per la valutazione.
2. **Parte applicativa:** diagrammi **UML** e design patterns per la progettazione del software.
3. **Stima dell'effort richiesto:** a livello di costi e di tempo, tipicamente suddividendolo in sottoproblemi e valutandoli.

- **Implementazione:** in base alla completezza della progettazione posso iniziare subito col codice o fare un po' e un po'.

1. **Implementazione:** scelta delle convenzioni, dei nomi e delle maiuscole, in modo che siano significativi.
  - (a) Conoscenza della standard library per risparmiare tempo ed eseguire test di unità e mostrare il codice al altri.

- **Testing/Debug:** utilizzare test case per vedere se ci sono errori e tramite Debug risolverli e ricominciare il testing.

1. **Testing:** va fatto sia durante lo sviluppo che al completamento.
  - (a) **Black box testing:** non viene guardato il codice, ma i requisiti (quindi tendenzialmente fatta dal cliente BETA).
  - (b) **White box testing:** viene testato con valori specifici conoscendo il codice (usando valori critici ALPHA).

- **Distribuzione e manutenzione.**

# Capitolo 2

## Costruire un programma

In un qualsiasi programma, ci sono alcune cose da definire e considerare. Infatti, possiamo a tutti gli effetti considerarlo un sistema.

### 2.1 Dimensione e Complessità

Si possono definire due grandi problemi per la dimensione e la complessità:

1. **Breadth issues** (problemi di ampiezza):

- **Funzioni principali**, generiche di cosa voglio che faccia.
- **Features** all'interno di ogni funzione.
- **Interfacce** verso sistemi esterni.
- **Utenti** connessi simultaneamente.
- Tipi di **dati e strutture** in cui vengono raccolti.

2. **Depth issues** (problemi di profondità):

- **Collegamenti e relazioni** tra gli oggetti: condivisione dei dati e di controllo, gerarchie, sequenziale, iterativo, ricorsivo.

#### 2.1.1 Divide et Impera

Si deve suddividere il problema in problemi minori più facilmente risolvibili.

- **Decomposizione.**
- **Modularizzazione.**
- **Separazione**
- **Iterazioni** incrementali.

Quando si lavora in team sullo sviluppo di un progetto complesso bisogna mettersi d'accordo su vari aspetti:

- Linguaggio di programmazione, tecnologie e ambiente di sviluppo.
- Sistema di controllo di versione, suddivisione e implementazione di task anche collegati.

### 2.2 CI/CD (Continuous Integration/Continuous Deployment)

Ogni task una volta completata viene integrata e testato tutto. Alcuni progetti diventano un fallimento perché è difficile stimare l'**effort** reale per sviluppare tutte le **features** richieste. Quando si passa all'integrazione di una nuova **features** è necessaria la comunicazione tra il team e a volte può fallire. Quando si gestisce un sistema ampio è necessaria la coordinazione tra le **3P** perché c'è un importante **effort** di coordinazione:

1. **Processi** e metodologie utilizzate: migliorare la qualità del software e la produttività degli sviluppatori.
2. **Prodotto** finale e artefatti intermedi: al giorno d'oggi non basta consegnare l'eseguibile finale ma anche documentazioni e altro.
3. **Persone** (sviluppatori, personale di supporto e utenti): coordinazione di queste ultime e attento studio sulle skill di ciascuno.

## Capitolo 3

# Ingegneria del Software

Per definizione:

- È una **disciplina** dell'ingegneria che riguarda tutti gli aspetti del software dai requisiti fino al mantenimento futuro.
- **CMU** dice che sia una forma di ingegneria che applica i principi dell'IT per ottenere una soluzione costi-efficace.
- **IEEE 610** invece la considera un approccio sistematico, disciplinato e quantificabile allo sviluppo e mantenimento.

Un progetto viene definito un successo che rispetta tre parametri di misurazione:

1. La consegna è stata effettuata in **tempo**.
2. Il progetto è rientrato nei **costi** preventivati.
3. Ha prodotto un **output** che ha soddisfatto il cliente.

Se uno di questi non è stato rispettato il progetto è **challenged**, se nessuno si dice che ha **fallito**. Tendenzialmente i progetti hanno successo quando coinvolgono l'utente, hanno richieste chiare e sono pianificati bene. Altrimenti falliscono soprattutto per mancanza di input, chiarificazioni delle specifiche o cambiamento in corso d'opera:

1. Buon **team di sviluppo**: buone skill, coordinamento, influenza, problem solving e confronto.
2. Buona **pubblicità**: visione chiara del progetto, tempi decisionali ristretti, disponibilità.
3. Buoni **spazi di sviluppo**: coinvolgimento dell'utente, ottimizzazione, esecuzione veloce.

Gli **errori** sono generalmente generati dal codice, ma può capitare che si siano propagati da delle specifiche non chiare. Per ridurre tali **bug** si preferisce nei progetti ampi utilizzare parti di codice già sviluppati.

*Un ingegnere del software è una costruzione multipersonale di un software multi versione.*

Eticamente per l'ingegneria del software ci sono **8 principi**:

1. Pubblico interessato.
2. Cliente e Datore.
3. Prodotto.
4. Giudizio sull'integrità integrità del prodotto.
5. Gestione del processo etico.
6. Professionalità.
7. Colleghi.
8. Pratica e continua formazione.

Tali principi vengono utilizzati per la privacy, la proprietà intellettuale, il riuso del codice, la risoluzione dei problemi.

**Alan Davis** nel **1994** ha definito alcune delle regole per sviluppare un software professionale:

1. Qualità al primo posto, dato che è possibile fare software di qualità.
2. Fornire il prodotto al cliente il prima possibile.
3. Inquadrare il problema prima di iniziare a segnare le specifiche.
4. Valutare eventuali alternative di progettazione.
5. Usare un appropriato modello di processo.
6. Usare linguaggi differenti per diverse fasi dello sviluppo.
7. Mettere le tecniche davanti agli strumenti per applicarle.
8. Preferire fare le scelte giuste, rispetto che quelle più veloci.
9. Una buona gestione è più importante di una buona tecnologia.
10. Le persone sono la chiave del successo, quindi prendersi responsabilità e seguire il progetto con cura.

**Anthony Wasserman** nel **1996** ha proposto alcuni concetti base per tale disciplina:

1. **Astrazione:** una tecnica utilizzata per la progettazione che permette di generalizzare concetti.
2. **Analisi e metodi/notazioni di sviluppo:** standardizzare i requisiti e le notazioni per comunicare universalmente.
3. **Prototipare la UI:** permette di inquadrare le preferenze dell'utente prima di iniziare lo sviluppo.
4. **Modularità e architettura:** permette di aumentare la qualità e la manutenibilità del software.
5. **Riutilizzo:** è una pratica nell'industria software per permettere di allungare la vita del progetto.
6. **Ciclo di vita:** deve essere considerato dall'inizio alla fine.
7. **Metriche di processo:** permette di verificare spesso l'andamento del software.
8. **Strumenti e ambienti integrati:** essenziali per sviluppare software complessi.

## 3.1 Processi Software

Per sviluppare un buon software c'è bisogno di varie **task** svolte da diverse persone e coordinate da un terzo:

- **CODE N FIX:** modello più semplice di programmare da solo che una volta soddisfatte le specifiche si fa test e debug.
- **WATERFALL:** modello a cascata che segue gli step principali a cui si aggiunge il mantenimento del software:
  - L'**output** del task precedente è l'**input** del successivo ed è utile per descrivere lo stato attuale del progetto.
  - Purtroppo l'interazione con l'utente avviene solo nella fase requisiti ed è **document-driven** perché ne fa molti.
- **INCREMENTALE:** ogni feature viene sviluppata separatamente poi unita tramite integrazione e testata:
  - I vantaggi sono sviluppare per primi i componenti più importanti e contenere il rischio se c'è un problema.
  - Può essere utilizzato per sviluppo **multi-componente**, oppure **multi-release** tramite varie versioni del prodotto.
  - In pratica utilizza il metodo **Divide and Conquer** ed è precursore della tecnica attuale **CI/CD**.

## Capitolo 4

# Ingegneria dei Requisiti

**Cos'è:** serie di attività legate alla convalida dell'insieme finale dei requisiti del programma.

Fra queste attività:

- **Preparazione:** concordare ed acquisire le risorse necessarie per pianificare il processo di ingegnerizzazione dei requisiti.
- **Elicitazione.**
- **Definizione e documentazione.**
- **Prototipazione.**
- **Analisi dei requisiti.**
- **Controllo e validazione.**
- **Specificazione requisiti.**
- **Accettazione requisiti.**

### 4.1 Elicitation Process

I **requisiti** devono riflettere le necessità del cliente e coinvolgere anche il management non solo l'immaginazione del team:

- **Elicitazione di alto livello:** comprendere la relazione con il business e le giustificazioni per il software.
  - Profilo dell'azienda a livello di necessità, giustificazioni, scopi, vincoli, funzionalità, fattori di successo e utente.
  - Con questo si intende budget per distinguere i *nice-to-have*, scadenza, necessità quotidiane degli utilizzatori.
- **Elicitazione di dettaglio:** ottenere informazioni dettagliate riguardo necessità e desideri dell'utente.
  - Questi metodi possono comprendere incontri verbali, report scritti, form da compilare online.

Una volta che il piano è stato approvato si possono decidere le **risorse** per lo sviluppo.

Nell'ottenere informazioni sul progetto si preferiscono interazioni orali, anche riguardo politiche tecniche e manuali tecnici. Ci sono sei dimensioni dei requisiti:

1. **Flusso di lavoro dell'azienda (BF).**
2. **Formato della data e dei dati (DF).**
3. **Funzionalità individuali (IF).**
4. **Preesistenza di interfacce e sistemi:** controllo software, trasferimento dati, gestione risposte, possibilità di riprovare (**IS**).

5. **Interfaccia utente:** tramite prototipazione (**UI**).
6. **Performance, affidabilità e altri vincoli:** riguardano il dominio dell'applicazione (transazioni, lunga vita, finanziaria) (**FC**).

## 4.2 Analisi dei Requisiti

L'analisi dei requisiti ha il compito di **categorizzarli** o raggrupparli e poi assegnare a ciascun gruppo una **priorità**:

- Posso fare la scelta di **categorizzare** i requisiti usando lo schema delle sei dimensioni descritto in precedenza.
  - I requisiti possono essere limitati da alcuni vincoli come ad esempio poche risorse, poco tempo, poche competenze.
- Per stabilire le **priorità** si considerano le richieste attive, la competitività, la possibilità di vendita e i problemi futuri.
  - Di solito si usa un processo di gerarchia analitico (**AHP**) dove ogni requisito è comparato a tutti gli altri con dei valori di intensità.
- Gli **use-case** orientati ad oggetti aiutano molto a descrivere i requisiti e analizzare i sistemi.
  - Solitamente includono le funzionalità di base, precondizioni e postcondizioni, condizioni di errori e flussi alternativi.
  - Nel processo di creazione bisogna individuare gli attori coinvolti, gli use-case collegati e le condizioni di confine del sistema.
- In alternativa per l'analisi posso utilizzare il modello **FURPS+** che è un acronimo che si articola in:
  - **Funzionalità.**
  - **Utilizzabilità.**
  - **Affidabilità. (Reliability).**
  - **Performance.**
  - **Compabilità.**
  - **+ Requisiti di design, implementazione, interfaccia e fisici.**

## 4.3 Definizione, Prototipazione e Review

Ogni requisito segna cosa vuole il **cliente**, gli **input/output** il **processo** da attuare. Tramite opportuni formalismi, dopo avere prototipato i requisiti, possiamo descrivere tramite **standard IEEE** un sommario dei requisiti:

- Tra queste notazioni ci sono i diagrammi **UML** e in particolare gli **use-case**, **activity diagram**, **Data Flow Diagram**, **ER**.
- Il livello di **dettaglio** dipende dalla complessità del progetto, dall'attività di supporto, l'esperienza e le pre-release.

## 4.4 Requirement specification

Al termine dell'intero processo di rilevazione dei requisiti devono essere prodotti questi documenti informativi:

- **BRS:** specifiche del business tra i più importanti che contiene scopo del business, principali investitori, ambiente del business, mission, business model, processi, vincoli, policy e struttura del business.
- **StRS:** specifiche degli investitori che contiene le stesse cose del precedente ma declinate sugli investitori.
- **SyRS:** specifiche dei dispositivi coinvolti come sopra ma aggiunge la sicurezza del sistema, privacy, ciclo di vita, caratteristiche e packaging.
- **SRS:** il più utilizzato che parla del software che contiene anche le limitazioni, le prospettive del prodotto, interfacce, utilizzabilità, standard.

# Capitolo 5

## Design SOLID

### 5.1 Architettura e Design

Ogni software rappresenta due valori principali per l'investitore:

- **Comportamento**, cioè le funzioni principali che svolge.
- **Struttura**, cioè un programma facilmente modificabile, perché verrà modificato se è utilizzato.

Una volta che i requisiti sono chiari, si trasformano in **design** che ne definisce i componenti e le loro interazioni ed è diviso in due fasi:

1. **FASE ARCHITETTURALE**: visione di alto livello, proprietà esterne e relazioni, dettate dai requisiti funzionali e non.
2. **FASE DETTAGLIO DESIGN**: decomposizione dei componenti in dettagli, dall'architettura ed i requisiti funzionali.

### 5.2 Principi SOLID

- **Single Responsibility Principle (SRP)**, la classe deve avere un solo motivo per cambiare:
  - Deve avere una sola funzionalità, per evitare di creare un malfunzionamento in caso di cambiamento.
  - È difficile da implementare perché le classi diventano tante e perché siamo abituati a raggruppare le funzionalità.
  - Un modulo è un file sorgente che coinvolge più classi ad alta coesione che rispettano il principio **SRP**.

**SOLUZIONE:** ogni classe deve contenere il codice per la sola funzione che gli spetta ed utilizza il pattern **FACADE**.

- **Open Close Principle (OCP)**, deve essere possibile estendere il comportamento di una classe senza modificarla:
  - Ogni modifica su codice già rilasciato è vietata, bisogna creare un nuovo metodo o meglio una nuova classe.
  - Migliora la stabilità, manutenzione e riutilizzabilità del codice; aperta per implementazioni, chiusa per modifiche
  - Le variabili sono private, non si usano globali. Le frecce piene significano implementazioni, vuote invece relazioni. Se una freccia va dalla classe A alla B, significa che all'interno della classe A viene menzionata e usata la classe B.
  - In pratica utilizzo un'interfaccia ed una o più implementazioni con polimorfismo per definire un modulo ad alto livello, protetto.

**SOLUZIONE:** l'astrazione creando un interfaccia, i cui oggetti istanziati saranno attributi della classe originaria scelti con i setter.

- **Liskov Substitution Principle (LSP)**: la classi derivate devono essere sostitutive della loro classe base senza problemi:
  - La validità di un modello dipende dall'utilizzatore, dato che il focus è il comportamento pubblico della classe OOP.

**SOLUZIONE:** delegare i metodi alle sottoclassi che ereditano da quella base che contiene le firme delle funzioni.

- **Interface Segregation Principle (ISP)**, creare gerarchie di interfacce, in modo che il client non sia costretto a dipendere da metodi non usati:
  - Evitare che si utilizzino interfacce troppo ricche di metodi, perché potrebbero essere inutili per molte sottoclassi.
  - In caso di modifiche ricompilo solo le interfacce modificate, analogamente riduco le dipendenze alle essenziali

**SOLUZIONE:** creo più interfacce legate tra di loro, ognuna che specifica maggiormente la precedente, non una.

- **Dependency Inversion Principle (DIP)**, dipendere da astrazioni (stabili e non modificabili), non da classi specifiche:
  - Se un modulo di alto livello dipende da uno di basso livello non va bene! (esempio di pizza/pizzeria).
  - Deve dipendere da un'interfaccia, non da una concretizzazione della stessa.
  - **Dependency Injection**, detta anche inversione di controllo, configura le dipendenze dall'esterno non staticamente. Iniettare tramite parametro nel costruttore, tramite un metodo setter esposto o tramite interfaccia con un iniettore.

**SOLUZIONE:** introdurre un livello di astrazione tra alto e basso livello, che dipendono da un'interfaccia comune.

## 5.3 Principi di coesione dei componenti

I primi due sono inclusivi, mentre il terzo esclusivo e sono tutti essenziali per evitare sbilanciamenti. Un componente è l'unità di codice più piccola che può essere distribuita e compilata indipendentemente, che devono rimanere coesi quando scelgo le classi:

- **Reuse/Release Equivalence (REP)**: rilascio associato ad un numero e comporta la coerenza dei componenti nei pacchetti che vengono distribuiti.
- **Common Closure Principle (CCP)**: raccolgo in componenti le classi che possono cambiare per la stessa ragione, analogia con il principio solid **SRP**.
- **Common Reuse Principle (CRP)**: non costringere l'utilizzatore a dipendere da codice di cui non ha bisogno, analogia con **ISP**.

## 5.4 Principi di accoppiamento dei componenti

Ci permettono di gestire le relazioni tra i componenti:

- **Acyclic Dependencies Principle (ADP)**, impedire cicli nel grafo delle dipendenze perché se si lavora in team si rischia di rompere il lavoro degli altri.
- **Stable Dependencies Principle (SDP)**, si cerca di dipendere nella direzione della stabilità ed è importante capire i componenti stabili e instabili.
- **Stable Abstraction Principle (SAP)**, un componente deve far dipendere il suo livello di astrazione dal suo livello di stabilità.

# Capitolo 6

## Architettura del software

**Cos'è:** si tratta dell'organizzazione del sistema o di come i componenti principali ad alto livello sono connessi tra di loro.

### 6.1 Elementi dell'architettura del software

1. **Caratteristiche architetturali:** descrivere le qualità del software.
  - (a) Le caratteristiche architetturali sono i requisiti non funzionali, attributi per la qualità del sistema (scalabilità, affidabilità, manutenibilità).
2. **Decisioni architetturali:** scelte sulla struttura del sistema che hanno un impatto significativo nel tempo.
  - (a) Diventano vincoli che guidano lo sviluppo del software, documentati soprattutto nei progetti più grandi.
3. **Componenti logici:** i blocchi di costruzione di un sistema che incapsulano funzionalità specifiche, per esempio tramite directory.
4. **Stili architetturali:** la forma e struttura generale del sistema software, definendo come i componenti sono organizzati tra di loro.
  - (a) Ad esempio i microservizi, i layer, event-driven.

L'**architettura** è ad **alto livello** ed ha impatto sul lungo termine, mentre invece la **progettazione** definisce la struttura di ogni classe e come interagiscono. L'architettura riguarda la struttura del sistema, i servizi, il DB e la interazione UI, mentre il design riguarda come le funzionalità sono implementate. Si distinguono decisioni strategiche a **lungo termine** o tattiche che sono a **breve termine** indipendenti dalle altre. Cambiare l'architettura è più costoso.

#### 6.1.1 Caratteristiche Architetturali

Derivate dal dominio del problema e dai requisiti, dall'ambiente di sviluppo, e dalla conoscenza del dominio di lavoro. È la parte che richiede più lavoro ed è inerente all'ambito del progetto e tendenzialmente sono **indipendenti dal dominio** di lavoro, si tratta infatti di capacità di svolgere certe operazioni, **decisioni** strutturali interne, o sulla **qualità**. Influenzano l'**architettura strutturale** e sono legate al successo. Possono essere **esplicite**, ovvero specificate direttamente nei requisiti, oppure **implicite** cioè dedotte o necessarie dal contesto.

- **Processo:** descrive dove il processo di sviluppo software interseca l'architettura dello stesso. Comprende modularità, facilità di testabilità, efficienza di distribuzione, facilità di estensibilità e capacità di disaccoppiamento positiva.
- **Strutturali:** ciò che impatta la struttura interna compreso l'accoppiamento e le relazioni di integrazione. Comprende sicurezza, manutenibilità, portabilità multiplatforma, facilità di estensione del software, localizzazione con supporto multi-lingua.

- **Operazionali:** come le decisioni architetturali impattano sulle capacità operative del sistema. Comprende disponibilità, ripristinabilità dopo un'interruzione, robustezza in casi limite, tempo e performance, affidabilità, scalabilità utente.
- **Cross-Cutting** decisioni che impattano sul sistema nel suo insieme. Comprende sicurezza generale, aspetti legali e leggi su dati e altro, autenticazione e autorizzazione (AAA), privacy e crittografia delle transazioni, accessibilità per tutti gli utenti, utilizzabilità.

### 6.1.2 Decisioni Architetturali

Per la comunicazione tra servizi si può usare il design pattern **Observer**, che si può implementare in due modi diversi, tramite una **Queue** oppure con **Topic**. Per tenere traccia delle decisioni si utilizzano gli **ADRs** (**Record di Decisione Architetturali**), ovvero un insieme di campi che compiliamo per ogni decisione.

### 6.1.3 Componenti Logici

Si tratta di **blocchi funzionali** del sistema all'interno del proprio dominio di lavoro. **Architettura logica** significa quindi rappresentare l'interazione tra componenti, mentre **fisica** specifica come il sistema viene realizzato nella pratica.

- Identificare i **componenti core**: tramite un approccio di analisi di workflow, oppure analizzando le operazioni svolte dagli attori.
- Assegnazione dei **requisiti** al giusto componente che è responsabile di adempirli, evitando di creare entità che non rispettano il **SRP**.
- Analisi dei **ruoli** e delle **responsabilità**: assicurarmi che il componente faccia solo quello che è stato chiesto e verifica della coesione interna.
- Analisi delle **caratteristiche architetturali** per confermare che il componente si trova allineato con una delle caratteristiche chiave per il **successo**.

**ACCOPPIAMENTO** dei componenti si intende il grado di dipendenza tra di essi e la conoscenza che hanno gli uni degli altri, se eccessiva difficile da mantenere.

**AFFERENZA/EFERENZA:** fan-in/fan-out si tratta del numero di classi che dipendono dal componente e il numero di classi da cui il componente dipende.

### 6.1.4 Stili Architetturali

Possono essere caratterizzati da partizionamento tecnico o di dominio, oppure modello di distribuzione.

|                  |             | Partitioning           |                  |
|------------------|-------------|------------------------|------------------|
|                  |             | Technical              | Domain           |
| Deployment model | Monolith    | Layered<br>Microkernel | Modular monolith |
|                  | Distributed | Event-driven           | Microservices    |

Figura 6.1: Tabella con gli stili architetturali.

Può essere **monolitico** se si basano su un'unica unità, altrimenti **distribuito** se è organizzato in componenti che possono essere provati in luoghi diversi. Il partizionamento **tecnico** divide il software in pacchetti, mentre quello per dominio come dice il termine è suddiviso in moduli legati al **dominio**.

- Il **monolitico** è semplice, economico, più facilmente realizzabile e debuggabile, ma non è scalabile, difficile da evolvere e va ridistribuito per intero.
- I sistemi **distribuiti** invece sono più scalabili, modulari, resilienti, testabili e distribuibili più facilmente. D'altra parte però sono più lenti, difficili da debuggare e risalire allo stack di errore, sono più complessi e costosi.

## Monoliti a Livelli

Principalmente utilizzati quando deve entrare in produzione velocemente, oppure quando è importante separare le responsabilità rispettando **SRP** per i lavoratori che abbiamo, o ancora quando abbiamo bisogno di un prodotto facilmente estensibile. Utilizza il paradigma di programmazione **MVC** ovvero **Model View Controller**. Il modello rappresenta la logica di business, la vista rappresenta l'interfaccia utente, il controller si occupa del workflow e delle funzionalità coese.

I livelli partizionati sono **isolati** dagli altri, **indipendenti**, **chiusi** tra di loro e sono facilmente **rimpiazzabili**. Tuttavia esiste il modo di lasciare alcuni livelli **aperti**, creando un livello servizi quando ci sia la necessità di avere oggetti condivisi all'interno del livello di business. A livello di **architettura fisica**:

1. Architettura fisica a **due strati** che separa la parte della memorizzazione dai dati dal resto, è semplice, veloce e a scalabilità e affidabilità media.
2. Architettura fisica a **tre strati** che separa anche la parte di presentazione web ma è la meno affidabile perché complessa nonostante sia distribuita.
3. Architettura fisica di tipo **embedded o mobile** che contiene tutto, è il meno scalabile, limitato dalle risorse e legato molto all'implementazione.

Questo tipo di partizionamento è velocemente **realizzabile**, **efficiente** e **ottimizzato**. Non è, però, scalabile, è difficile da testare e distribuire e rischioso.

## Monoliti Modulari

Permettono di eseguire una partizione funzionale in base al **dominio** dell'operazione, o al **sottodominio**. Dividono il progetto in moduli, ciascuno dei quali ha tre componenti di presentazione, logica di business e persistenza dei dati. I team hanno consapevolezza degli interi livelli di quel dominio. Per mantenere i moduli modulari bisogna mantenere le classi disaccoppiate in modo da evitare delle dipendenze indesiderate. I moduli sono considerabili ciascuno un **servizio indipendente** ma distribuiti come uno unico, ciascuno che espone le proprie **API** e nasconde l'implementazione.

## Micorkernel

Permette di aggiungere i **plugin** al **core** del sistema, tramite un registro di plugin usando dei contratti per eseguirli. I plugin possono essere di tipo **monolitico** ed essere **embedded** con il core, oppure **distribuiti** ed essere anche su altre tecnologie e dispositivi **sync** o **async**. Il core contiene un **registro** che contiene la lista dei plugin disponibili per l'esecuzione e come accedervi, quindi contratti e protocolli.

- I **contratti** possono essere standard e descrivere caratteristiche comuni, oppure personalizzati e spesso richiedono un adapter per convertirli.

Viene utilizzato moltissimo perché è **scalabile**, **adattabile**, **evolvibile**, ha una struttura semplice e con il partizionamento garantisce **flessibilità**. Gli svantaggi avvengono quando devo modificare spesso il core, quando introduco dipendenze tra i plugin o risorse condivise, ed è meno performante.

## Microservizi

Un **microservizio** è un'unità di codice single-purpose, distribuita indipendentemente.

Le architetture di questo tipo sono distribuite, partizionate per dominio.

- **Granularità**: è una delle caratteristiche più importanti perché se facciamo servizi troppo piccoli serve coordinazione, se troppo grandi perde di senso.
  - **Disintegratori**: forzano a fare servizi piccoli quando il codice è volatile, può fallire, quando è scalabile o serve **ACL**.
  - **Integratori**: forzano a fare servizi grossi, quando ci sono dipendenze di dati o tra microservizi, o in fase di commit sul **DB**.

Il **riutilizzo** del codice in questi casi è essenziale per funzioni comuni a tutti come i logger e posso far dipendere il codice da librerie comuni più veloci, meno rischiose, ma complesse, difficili da mantenere e distribuire; oppure da servizi comuni rischiosi per errori di rete, ma implementabili in qualsiasi linguaggio.

Per il flusso di lavoro e la coesione dei microservizi posso usare due metodi:

1. **Orchestrazione:** un broker che stabilisce la successione delle operazioni, in cui è più facile da cambiare l'ordine.
2. **Coreografia:** ogni servizio chiama il suo successore, migliori performance, scalabile ma più difficile da debuggare.

La struttura a microservizi è più facile da **mantenere, testare, distribuire, evolvere**, è **scalabile e tollerante** agli errori. D'altra parte invece è scomoda perché più **complessa**, accoppiata al database in ogni servizio, per **problemi di rete** e richiede più team tutti **competenti** su tutto.

### Event-Driven

**Eventi:** sono avvenimenti importanti che vengono inoltrati in broadcast ai servizi listener e solo quelli interessati agiscono. Le comunicazioni sono asincrone, dato che chi genera l'evento non attende risposta, difficile la gestione dell'errore. I **messaggi** invece sono comandi diretti.

Ci sono diversi livelli su cui valutare questo tipo di architettura:

- **Performance:** gli **EDA** permettono il parallelismo e l'asincrono quindi sono più veloci dei microservizi che introducono latenza.
- **Dipendenza fisica:** gli **EDA** non hanno bisogno di esserlo, mentre i microservizi richiedono la proprietà dei dati, non la condivisione.
- **Granularità dei dati:** i microservizi possiedono ciascuno i propri, mentre negli **EDA** si possono avere varie strutture **DB** che vedremo.
- **Granularità dei servizi:** i microservizi sono molto specifici e fanno una funzione, mentre gli **EDA** possono essere entrambi.
- **Processamento:** gli **EDA** si basano sull'elaborare eventi e reagire ad essi, mentre i microservizi elaborano su richiesta.
- **Comunicazione:** gli **EDA** comunicano asincronicamente tra servizi, mentre i microservizi sono sincroni e usano **Rest**.

Tra i **vantaggi** di un'architettura **EDA** c'è il disaccoppiamento per migliorare la **manutenzione**, la **velocità**, resistenza agli **errori** e **scalabilità**. Gli **svantaggi** invece includono l'**accoppiamento** con i **DB** o tra i **servizi**, la difficoltà di **testing**, **inadeguatezza** a chiamate sincrone, e la **complessità**.

La **topologia** del database può essere di tipo **monolitico**, con un unico DB molto centralizzato. Oppure **partizionata** per dominio, dove ogni dominio ha il suo DB ma introduce accoppiamento tra servizi o ogni servizio ha il suo DB.

- Il **modello broker** ha un semplice gestore degli eventi leggero su ogni **listener** che accetta dal broker l'evento, svolge una task e risponde asincrono.
  - L'evento iniziale avvia tutto il processo, viene inviato al broker degli eventi, che poi riceverà e inoltrerà l'evento processato.
  - I vantaggi sono le performance, la scalabilità, la responsività, ma è difficile gestire gli errori, controllare il workflow e ripristinare backups.
- Il **modello mediatore** ha un mediatore di eventi centrale che gestisce e controlla il flusso di lavoro e deve coordinare gli event processor.
  - Coinvolge quindi una un evento iniziatore, una coda di eventi, che passano per il mediatore e sul canale eventi per essere processati.
  - Un possibile miglioramento può essere inserire più mediatori, uno per ogni dominio per evitare di aver un singolo punto di fallimento.
  - I vantaggi sono il controllo, la gestione degli errori, il ripristino, ma ha minore scalabilità per il mediatore, basse performance e accoppiamento.

## Capitolo 7

# Architettura Pulita

**Obiettivo:** creare un'architettura per massimizzare la produttività dividendola in componenti che comunicano.

**Scopo:** facilitare comprensibilità, sviluppo, distribuzione, operatività e mantenimento quindi supportando l'intero ciclo di vita lasciando opzioni aperte.

- **Sviluppo:** un software che ha uno sviluppo difficile non avrà vita lunga.
  - **Distribuzione:** un software che è costoso da distribuire e compilare sarà poco utile.
  - **Operatività:** non è molto influenzata all'architettura perché c'entra l'hardware ma dovrebbero Sfruttarla per renderle leggibili allo sviluppatore.
  - **Mantenimento:** il costo principale del mantenimento è la *speleologia*, ossia cercare dove aggiungere nuove funzioni, e il rischio di rompere tutto.
- **Disaccoppiamento dei livelli:** si usano **SRP** e **CCP** separando la gestione UI, dalle regole di business dipendenti (e non) dall'applicativo e il livello del DB.
  - **Disaccoppiamento use-case:** si tratta di creare fette verticali sui livelli, in modo da separare porzioni di codice che cambiano in base alla funzionalità.

### 7.1 Livelli di Disaccoppiamento

1. **Livello sorgente:** pone l'attenzione sull'evitare dipendenze nei moduli centrali in modo da evitare di ricompilarli tutti per ogni modifica.
2. **Livello di distribuzione:** pone l'attenzione sull'evitare dipendenza tra unità distribuibili che devono essere indipendenti l'un l'altra.
  - (a) I componenti risiedono nello stesso spazio indirizzi e utilizzano chiamate a funzioni, comunicando con **IPC**, **socket** o **memoria condivisa**.
3. **Livello servizi:** le dipendenze sono ridotte alla comunicazione livello dati, mentre i componenti comunicano tramite pacchetti rete e sono indipendenti.
  - (a) Quest'ultimo è tendenzialmente costoso, per cui si inizia da un **monolite**, poi si divide in **unità** distribuibili e si espandono in **servizi** o **microservizi**.

È importante tracciare **confini** per decidere quali componenti fanno dell'esistenza altrui o dipendono da essi, ma le decisioni non devono essere premature.

**Policy:** regole per trasformare gli input in output che possono essere suddivise in calcoli, formattazione e validazione input.

**Livello:** la distanza tra gli input e gli output.

**Regole di business:** procedure che esistono al di là della loro digitalizzazione e possono essere critiche per il business usando dati critici.

**Entità:** oggetti che nascono dai legami intrinseci tra le regole critiche e i loro relativi dati.

## 7.2 Architettura Pulita

Ogni cerchio concentrico rappresenta aree diverse del software, quelli interni sono le policy, quelli esterni i meccanismi:

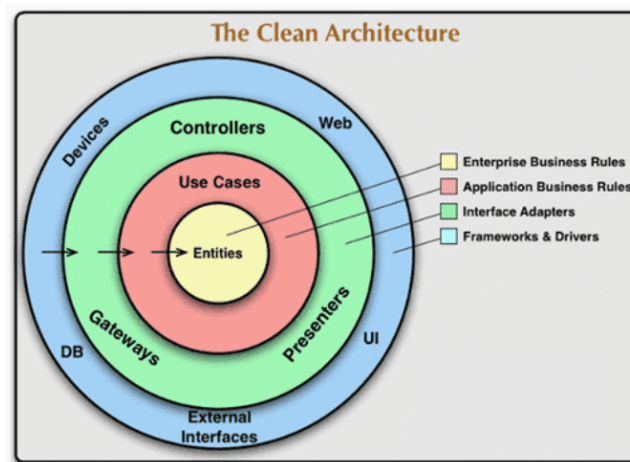


Figura 7.1: Grafico di Architettura Pulita.

Dove:

- **Entità:** è il cerchio più interno per cui è quello di più alto livello su cui dipende tutto e contiene le regole critiche di business riutilizzabili.
- **Use-Case:** cerchio centrale che contiene le regole legate all'applicativo e gestisce il flusso di dati da/a entità e le modifiche qui non toccano le entità.
- **Interface Adapters:** converte i dati per passarli ai servizi esterni con adapter, contiene tutti i componenti MVC.
- **Framework e Driver:** contiene DB, framework web e altri strumenti e contiene codice minimale che dipende dagli adapters.
- **Main Component:** componente più esterno e di basso livello che definisce strategy, factory e utilizza un framework di iniezione delle dipendenze.

### 7.3 Documentazione 4+1

- Una vista **logica** che mostra le astrazioni chiave del sistema.
- Una vista di **processo** che mostra l'interazione in fase run time dei processi.
- Una vista di **sviluppo** che mostra come il software è decomposto per lo sviluppo.
- Una vista **fisica** come il software è distribuito tra i processori dell'hardware.
- Documentazione aggiuntiva su altri scenari o use-case.

## 7.4 Modello C4

- **System context diagram:** è una vista che contiene il sistema stesso e le persone che comunicano con esso.
- **Container diagram:** contiene tutti i container, come interagiscono tra loro e con le persone.
- **Component diagram:** componenti, responsabilità, implementazioni e interazioni di un container.
- **Code:** decompone un componente e mostra come è implementato ed è opzionale e sconsigliato.

## Capitolo 8

# Metriche di Qualità

Il codice sorgente non è l'unico **artefatto** da valutare quando si verifica la bontà del codice. L'**analisi dei requisiti** è uno degli aspetti su cui posso valutare la qualità del software:

- **Consistenza** nella progettazione.
- **Completezza** nella progettazione.

L'obiettivo delle **metriche** è quello di valutare attraverso il **ciclo di vita** del software come questo si conforma a certe richieste di qualità. Riduce la soggettività, anche se comunque richiede il giudizio di un essere umano.

### 8.1 Metrica Software

**Metrica Software:** è una misura quantitativa su cui il sistema di software, i componenti o i processi hanno una proprietà o caratteristica.

1. Metriche di **prodotto**: sul prodotto finale o intermedio sia statico che runtime.
2. Metriche di **processo**: riguardano metodi, tecniche e strumenti impiegati nello sviluppo e mantenimento del sistema.
3. Metriche di **progetto**: riguardano il progetto intero.

#### 8.1.1 Misure Statiche

Valutano il software senza eseguirlo e sono misure facilmente definibili:

- **LOC (Lines of Code)**, esistono anche SLOC, CLOC, NCLOC, LLOC, BLOC sono variazioni eventuali del numero di linee facili da misurare.
- **Duplicazione** del codice: evitare codice simile scritto più volte che andrebbe mantenuto contemporaneamente ogni modifica e non è praticabile. Esistono tecniche per rilevarla tramite match sintattici, o token match:
  - Duplicazione del primo **tipo**: spezzoni di codici che sono identiche tranne per formattazione e spazi.
  - Duplicazione del secondo **tipo**: spezzoni di codici che sono totalmente uguali tranne qualche variazione di token, variabili o primo tipo.
  - Duplicazione del **terzo** tipo: spezzoni uguali con l'aggiunta o la rimozione di qualche statement più duplicazione secondo tipo.
  - Duplicazione del **quarto** tipo: due porzioni di codice che fanno la stessa funzione ma implementato in modo diverso
- **Nesting Level (NL)**: livello di innesto del software basandosi sul conteggio delle linee di codice rispetto a un livello di innestamento. Utilizzata per individuare quali parti del codice sono più difficili da mantenere, aggiustare ma anche quelle che sarebbe meglio modificare.
- Metriche di complessità di **Halstead**: utilizza dei contatori  $n1$ ,  $n2$  per contare il numero di operatori e operandi distinti e  $N1$ ,  $N2$  per le occorrenze. Misurano il volume, l'effort, la difficoltà, la lunghezza del programma, il tempo richiesto, il numero di bug presenti.

- Complessità ciclomatica di **McCabe**: è il numero di cicli indipendenti. Misura la complessità della progettazione strutturale del programma e rappresenta il numero massimo di percorsi.

### Dipendenza fra moduli

Nelle metriche statiche viene anche presa in considerazione la dipendenza fra moduli, una **relazione** formale tra due pacchetti di **dipendenza**, quindi librerie in comune, comunicazione tra classi eccetera.

La metrica usata è **Henry-Kafura** che misura il **flusso intermodulare**, basato su **fan-in** e **fan-out** di una classe/pacchetto:

- **Accoppiamento**, il modulo fa riferimento a una dipendenza esterna ad esso, cioè dipendenza **intermodulo**. Ci sono vari gradi di accoppiamento:
  - Accoppiamento di **contenuto**: due unità accedono a dati o procedure interne altrui, difficile fare una modifica solo su una.
  - Accoppiamento **comune**: due unità accedono alla stessa variabile globale, comune in applicativi molto estesi.
  - Accoppiamento di **controllo**: un'unità passa un'informazione di controllo per influenzare il comportamento dell'altra.
  - Accoppiamento per **stampa**: un'unità passa un gruppo di dati ad un'altra, molto più di quelli necessari ottimali.
  - Accoppiamento **dati**: tra un'unità e un'altra vengono passati solamente i dati necessari, quindi bassa interdipendenza.

Si usa la misura di **Fenton-Melton** per assegnare un **grado**.

- **Coesione**, il modulo fa riferimento a dipendenze interne allo stesso, ovvero dipendenza **intramodulo**. Ci sono vari livelli di coesione:
  - Livello **funzionale**: svolge un'unica attività chiara e delineata, non viene modificato anche se non è molto flessibile.
  - Livello **sequenziale**: svolge un'attività principale, ma i confini di tale operazione non sono del tutto definiti.
  - Livello **comunicazione**: le attività svolgono operazioni sugli stessi dati e dimostra una vicinanza interna maggiore della procedurale.
  - Livello **procedurale**: i processi sono correlati in modo procedurale, connessi da qualche sequenza di controllo.
  - Livello **temporale**: gli elementi che vengono eseguiti nello stesso slot temporale vengono raggruppati, esempio l'inizializzazione.
  - Livello **logico**: il codice svolge una serie di compiti simili, cioè esiste una relazione tra gli elementi ma è molto debole.
  - Livello **coincidenza**: il codice svolge due o più funzioni non correlate ed è dovuto a modifiche successive nel codice.

Come misurazioni per la coesione del software si usano **Program Slice** e **Data Slice**, basati su **fette** di **dati** e **codice**.

### Metriche per l'OOP

Esistono metriche uniche per il linguaggio **OOP**, misurano se le classi che comunicano hanno basso **accoppiamento** e **coesione** forte.

Per esempio, le **Metriche CK** che misurano la **complessità** di progettazione e come impattano sulla **qualità** la **manutenibilità**, **riutilizzabilità** e altri:

- **Weighted Method per Class (WMC)**: somma pesata di tutti i metodi della classe, per esempio assegnando 1 ad ognuno.
- **Depth of Inheritance Tree (DIT)**: la profondità massima di dipendenze da una classe alla sua radice, ma ha un'interpretazione mista.
- **Number Of Children (NOC)**: il numero di sottoclassi di una classe, un alto NOC potrebbe aumentare la complessità e forse la qualità.

- **Coupling Between Object (CBO)**: il numero di classi a cui una classe è accoppiata, ma aumenta la difficoltà di manutenibilità.
- **Response For a Class (RFC)**: l'insieme dei metodi coinvolti nella risposta ad un messaggio inviato a un oggetto della classe.
- **Lack of Cohesion in Methods (LCOM)**: misura la correlazione tra i metodi e le istanze locali delle variabili di una classe.

### 8.1.2 Metriche Dinamiche

Valutano il software in esecuzione e sono più complesse perché analizzano il **comportamento**:

- **Copertura del codice**: quanto **codice** viene eseguito di tutto quello scritto.
- **Bugs-per-Line-of-Code**: numero di **bug** per linea di codice.
- **Utilizzo delle feature**: quanto le feature che sono state sviluppate risultano davvero **utili** all'utente.

## Capitolo 9

# Modelli e UML

**Sistema:** è un **insieme** di componenti che interagiscono e si influenzano tra loro, percepiti come unica unità per lo scopo comune.

**Astrazione:** **semplifica** la rappresentazione della **realtà** sia nelle generalizzazioni, sia nei processi, eliminando dettagli e identificando punti in comune.

### 9.1 UML

**UML** è una famiglia di notazioni grafiche per descrivere e progettare sistemi software, soprattutto quelli **OOP**. I linguaggi di programmazione sono troppo basso livello per discutere di progettazione software, i linguaggi grafici aiutano la comprensione di gruppo.

- **Sketch:** maggiormente utile per forward e reverse engineering enfatizzando la comunicazione selettiva piuttosto che completa.
  - **Blueprint:** enfasi sulla completezza e sui dettagli, che richiedono strumenti più sofisticati di quelli per costruire gli sketch.
  - **Linguaggio di programmazione:** disegnare dei diagrammi UML che vengono compilati in codice eseguibile.
- **Regole prescrittive:** per definire il corpo e specificare cosa è legale o illegale e quale significato è associato ad ogni frase del linguaggio.
  - **Regole descrittive:** per decidere come il linguaggio viene effettivamente utilizzato ed è quello più utilizzato nell'ambito **UML**.

È importante ricordare che se una specifica non è inclusa nell'**UML** non è detto che non sia presente nel progetto, perché contiene solo le cose utili. Esistono diversi tipi di modelli **UML**:

- **Class Diagram.**
- **Sequence Diagram.**
- **Activity Diagram.**
- **State Diagram.**
- **Use-Case Diagram.**

# Capitolo 10

## Implementazione

**Implementazione:** è l'atto di trasformare il **design** di dettaglio, spesso fatto in questa fase, in un **programma** in un certo linguaggio di programmazione. Una **buona** implementazione di un codice coinvolge **leggibilità**, **manutenibilità**, **performance**, **tracciabilità** a elementi di design, **correttezza**, **completezza**. Il programmatore si concentra subito sulla correttezza e le performance, ma è importante pensare prima a leggibilità e manutenibilità per avere gli altri.

### 10.1 Modello di Qualità del Prodotto Software

Composto da **8** caratteristiche, interne (**white box** in development) o esterne (**black box** in esecuzione):

1. **Idoneità di funzionalità:** la capacità del software di rispondere a certe esigenze in condizioni specifiche.
2. **Efficienza di performance:** il numero di risorse utilizzate sotto certe condizioni.
3. **Compatibilità:** il grado in cui due componenti comunicano dati tra di loro condividendo lo stesso hardware e software.
4. **Usabilità:** il grado di un sistema di raggiungere obiettivi di efficienza e soddisfazione utente in contesti reali.
5. **Affidabilità:** per quanto un sistema riesce a svolgere correttamente la sua funzione.
6. **Sicurezza:** la capacità di un sistema di proteggere le informazioni tramite livelli autorizzazione.
7. **Manutenibilità:** il grado di efficienza con il quale il prodotto può essere modificato.
8. **Portabilità:** l'efficacia con cui un sistema/componente può essere trasportato da un hw/sw a un altro.

### 10.2 Modello di Qualità d'Uso

Composto da **5** caratteristiche, definita quando viene testato in particolari contesti **reali** di utilizzo.

1. **Efficacia:** accuratezza e completezza con cui un'utente riesce a raggiungere certi obiettivi.
2. **Efficienza:** risorse utilizzate in relazione all'efficacia con cui l'utente raggiunge gli scopi preposti.
3. **Soddisfazione:** il grado di soddisfazione dei bisogni dell'utente in uno specifico contesto di utilizzo.
4. **Libertà dai rischi:** il grado con cui un prodotto mitiga i rischi potenziali alla salute e sicurezza, all'ambiente o ancora quelli economici e finanziari.
5. **Copertura di contesto:** la capacità di soddisfare tutti questi requisiti in modo coeso e anche in contesti non legati a quello analizzato.

## 10.3 Debugging

Processo molto iterativo per **localizzare** e **sistemare** gli **errori** nel codice, trovati tramite testing, che può suddividersi in 4 fasi:

1. **Stabilizzazione**: si tratta di riprodurre l'errore in un ambiente controllato per capire le condizioni di fallimento e formulare dei test case.
2. **Localizzazione**: identificare le parti del codice che producono l'errore analizzando log, tracce stack, variabili e tendenzialmente è la fase più lunga.
3. **Correzione**: il codice viene modificato nel luogo designato per risolvere l'errore, ma bisogna capirlo appieno per non introdurre nuovi difetti.
4. **Verifica**: si tratta di assicurarsi che l'errore sia sistemato senza averne creati altri (**regression test**) utilizzando i vecchi test.

Gli errori **sintattici** sono quando il programma viola le regole grammaticali del linguaggio, quelli **logici** sono quando produce risultati inaspettati o sbagliati.

## 10.4 Assertion and Defensive Programming

Si parla di **asserzioni** quando voglio utilizzare delle precondizioni e postcondizioni per verificare che il programma esegua correttamente, altrimenti sollevo un `AssertionError`. La sintassi è ***assert Expr : Expr***, dove la prima espressione è una condizione booleana, la seconda un messaggio se c'è l'errore.

- **Internal Invariants**: sono asserzioni utilizzate come controlli negli else o nei casi di default verificare che non ci siano condizioni impreviste.
- **Control-Flow Invariants**: vengono usate per indirizzare il flusso del programma in modo che non raggiunga casi vietati, utilizzando `assert false`
- **Precondizioni, Postcondizioni, Class Invariants**: per verificare prima o dopo l'esecuzione di qualcosa per evitare che succedano cose spiacevoli.
- **Loop Invariants**: asserzioni che verificano condizioni importanti per la giusta esecuzione del ciclo o del codice che viene dopo di esso.

Si parla di **programmazione difensiva** quando un programmatore è conscio del fatto che ci saranno dei **bug** per cui cerca di impedire il fallimento.

## 10.5 Ottimizzazione delle Performance

Spesso riduce la **leggibilità** e la **manutenibilità**, che sono la priorità tranne nei sistemi real time. Prima di modificare una funzione per ottimizzarla è bene verificarne i benefici, inoltre **modularizzare** il codice in modo da ottimizzare piccole porzioni.

**REFACTORING** quando vado a modificare la struttura interna del codice senza modificarne il comportamento esterno.

Sintomo della necessità di un refactoring è codice **duplicato**, metodi e classi **lunghe**, switch, **dipendenza** eccessiva, **violazione** dell'incapsulamento. Tra le tecniche di refactoring abbiamo l'estrazione di una classe o di un metodo da un blocco di codice, cambiare l'algoritmo, spostare un metodo.

## 10.6 Software Manutenibile

Tra le linee guida, scelte subito, conviene scrivere unità di codice **semplici** e **brevi**, **non duplicate**, che dipendono da **interfacce** concise e divise in **moduli**. I componenti di alto livello devono essere disaccoppiati, l'architettura dev'essere bilanciata e breve, con test e pipeline di sviluppo automatizzate:

- **Scrivere unità di codice brevi:** evitare di scrivere più di quindici linee perché è facile creare dei **test SRP**, semplici ed isolati ma anche da analizzare. Sono anche più facili da riutilizzare per ridurre la **duplicazione**.
  - Per evitare di avere funzioni estratte con una lista di parametri infiniti si preferisce inserire tutto in una **classe** con degli **attributi**.
- **Scrivere unità di codice semplice:** mantenere meno di quattro branch per unità per evitare manutenibilità, testabilità, modularità e predittività. Considerare McCabe per limitarne la complessità, dato che il numero di test esaustivi è il numero di branch aumentato di uno.
  - Per evitare catene di **if else** si rimpiazzano i **branch condizionali** con il **polimorfismo** o si sostituiscono condizioni innestate con **guard clause**.
- **Scrivere il codice una sola volta:** per evitare di copiare errori o doverli poi correggere in mille posti, magari dimenticandomeli e ritrovandoli dopo.
  - Per evitare di copia incollare con piccole variazioni estraggo le parti comuni, le sposto in una superclasse usando **SRP** e così posso evolverle.
- **Mantenere le interfacce brevi:** limitare il numero di parametri passati a massimo quattro, usando piuttosto degli oggetti **DTO** che li riuniscono.
  - Per evitare troppi parametri utilizzo il design pattern **Parameter Object** creando un **DTO** o più di una classe per raggruppare dati correlati.
- **Separare gli affari in moduli:** evitare moduli enormi, nascondendo l'implementazione con **interfacce** per migliorare il **disaccoppiamento** e la **leggibilità**.
  - Per evitare di riempire di interfacce, controllo che venga implementata almeno due volte e prima cerco se esiste la funzione nella std library.
- **Ridurre l'accoppiamento componenti:** si deve esporre all'esterno un'interfaccia con i metodi strettamente **necessari** soprattutto nei **top level**.
  - Per evitare l'accoppiamento bisogna **astrarre** bene, limitare la conoscenza interna, evitare codice di passaggio tra un componente e l'altro.
- **Mantenere bilanciata l'architettura:** bilanciare il numero e la dimensione dei componenti top level, mantenendolo tra sei e dodici, ognuno con **SRP**.
  - Organizzare i componenti per **dominio funzionale** e quindi ragionare sulla funzione che hanno per l'utente finale oppure per **specializzazione tecnica** cioè su come il sistema è creato per dividere le responsabilità ma difficile la comunicazione tra essi.
- **Minimizzare la dimensione dei sorgenti:** una **codebase** deve essere piccola. Un sistema può avere più codebase, ad esempio una **centrale** e più **plugin**, ma se sono lunghi aumento la densità d'errore ed il rischio di fallimento.
  - Per raggiungere l'obiettivo non devo duplicare codice, devo fare il **refactoring** di quello esistente, usare librerie terze e dividere il sistema.
- **Automatizzare i test:** tramite un buon **framework** apposito riesco ad incentivare il **refactoring** e introdurre un **testing** continuo nello sviluppo. Questo permette la ripetibilità dei test, rende efficiente lo sviluppo e il codice predicibile, fornisce documentazione del codice testato automatica.
  - Posso distinguere i test in **black box** e **white box**, come discusso successivamente e sono i modi per raggiungere questo obiettivo.
- **Scrivere codice pulito:** si tratta di evitare di lasciare brutti commenti, codice commentato o morto, numeri hardcoded, eccezioni non gestite.

# Capitolo 11

## Testing

È un'attività di verifica che permette di **controllare** che il software si comporti come lo sviluppatore e il cliente si aspettano. Coinvolge sia elementi **dinamici** che **statici**, perché verifica tutti gli **artefatti**.

- **Assicurazione di Qualità QA**, che permette di progettare il software in modo che a partire dal progetto allo sviluppo, ci siano pochi errori.
- **Controllo della Qualità QC**, una verifica finale che controlla che il codice sia esente da errori o ne abbia pochi.

**TESTING:** è un esame **sistematico** e **metodico** di un prodotto che sfrutta una varietà di tecnologie e deve mostrare che soddisfa lo scopo richiesto. Si esegue il software in dei casi di test in un ambiente simile a quello di **produzione**, che spesso non è la stessa macchina dello sviluppo. Posso far emergere l'errore tramite:

- **Ispezioni**, in cui dei tester verificano a mano che il codice sia scritto in modo corretto.
- **Metodi formali**, come tecniche matematiche per verificare che sia corretto.
- Librerie di **analisi statica** per evidenziare possibili errori.

**Unit Testing:** il test di granularità più piccola che valuta i componenti a livello individuale, eseguiti dai tester e partecipa anche l'utente con le beta.

### 11.1 Software Development Life Cycle (SDLC)

Si comincia dal raccogliere i requisiti, prosegue con il design e termina con l'implementazione, in tutte va testato:

1. **Test Statico:** senza dover eseguire il codice.
2. **Test Dinamico:** esecuzione del codice e verifica del comportamento.

Vengono testati i requisiti **funzionali** e **non funzionali**, rispettivamente basati sulle specifiche e in che modo le realizza.

- **Test funzionali:** controlla **completezza**, **correttezza** e **appropriatezza**, a partire dall'unità fino al quadro generale, normalmente **black box**.
- **Test non funzionali:** focalizzato sulle qualità e gli attributi comportamentali come **usabilità**, **performance**, **efficienza** e **sicurezza**, sia **black** che **white box**.
  - L'obiettivo è verificare la presenza di **problemi** di performance, valutando velocità, capacità, stabilità, scalabilità.
  - Testing di **carico** e **stress** test per valutarlo sotto stress.
  - Test di **usabilità** per valutare quanto tempo ci mette l'utente ad utilizzarlo.
  - Test di **sicurezza** per valutare la protezione dei dati; test di portabilità per vedere quanto è portabile in altri ambienti di utilizzo.

- Test di **conformità** se è conforme agli standard; test di configurazione per vedere se funziona più configurazioni; test di UI se è consistente.

Ci sono vari **livelli** e **fasi** di sviluppo del software che sono spesso correlate ad appositi test:

- **Component unit testing:** valutiamo la singola classe indipendente per valutare se soddisfa i requisiti funzionali, non funzionali per ridurre i rischi.
  - Per ognuno bisogna individuare le basi del test, gli obiettivi dello stesso, i difetti tipici che possono verificarsi, l'approccio e le responsabilità.
- **Integration testing:** dopo aver unito insieme le unità testiamo le interfacce tra componenti e le loro interazioni.
  - L'obiettivo è testare i sottosistemi e i database, API, microservizi per eliminare difetti di sequenza, comunicazione, codifica dei dati o errori.
  - A livello di sistema valuta le interazioni tra sistemi, pacchetti o servizi anche esterni. Spesso coinvolge tester, partner per il testing.
- **System testing:** verifica l'intero sistema integrato e che sia corretta l'architettura sottostante individuando le stesse cose degli altri test.
- **User Acceptance testing:** verifica che il sistema abbia raggiunto tutti i requisiti richiesti dall'utente per cui richiede la collaborazione dello stesso.
  - Tra le forme di test di accettazione si ha UAT per l'utente, OAT di tipo operativo, AT contrattuale e regolatorio, alpha e beta testing.

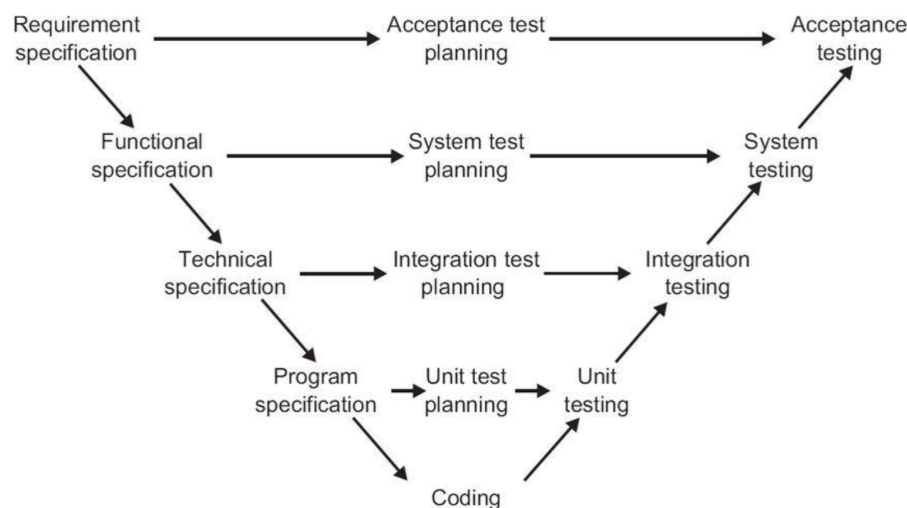


Figura 11.1: Schema processo di testing.

## 11.2 Componenti del Testing

Si divide in:

- **Test basis:** tutto il materiale utilizzato per il test.
- **Test condition:** condizioni che siano rispettate che vengono simulati, cerca di vagliare tutte le condizioni.
- **Test case:** si simula un comportamento o una operazione basata sulle prime due tecniche.
- **Test procedure:** sono il complesso delle attività che facciamo per portare a termine il test.

Per stabilire i test case posso usare **tecniche di testing**:

- Basate sull'**intuizione**: con esperienza e beta-testing, quando l'utente ha testato il prodotto senza vincoli avrò degli utenti esperti che insegnano.

- Basate sulle **specifiche**: con test **black-box**, senza esaminare il codice, che si basa sul comportamento del sistema con tecniche **ECP**, **BVA**.
- Basate sul **codice**: con test **white-box**, sfruttando la conoscenza del codice sorgente analizzando tutti i percorsi cercando la massima copertura.
- Basate su **test esistenti**: con **regression testing**, riutilizzando test precedentemente creati per verificare che non si siano reintrodotti i difetti.
- Basate sugli **errori**: tramite tentativi di indovinare basati sull'esperienza o analisi di porzioni di codice che tendenzialmente sono propense a difetti.

## 11.3 Processo di Testing

Diversi processi di test:

- **Test planning**: devo pianificare che cosa deve essere testato e quali sono gli obiettivi da raggiungere, per fare test organizzati, tracciabili, allineati.
- **Test monitoring e controllo**: stabilire delle misure di controllo e monitoraggio per vedere se sono raggiunti gli obiettivi tramite dei criteri d'uscita.
- **Test analysis**: esaminare le basi di test per identificare cosa c'è bisogno di testare, stabilendo una priorità di testing.
- **Test design**: scrivere i test case per valutare come bisogna testare ciò che è emerso con l'analisi, sfruttando le condizioni di test.
- **Test implementation**: comprendere come introdurre i parametri dentro il codice e scrivere le porzioni di codice per eseguirlo, creare l'ambiente.
- **Test execution**: eseguire nella pratica il test appena costruito e registrare i risultati.
- **Test completion**: terminazione della fase di testing per rilasciare una versione con un report degli errori corretti.

## 11.4 Processo di revisione

Si distingue la review **informale**, una discussione ad hoc da quella **formale**, tramite delle rigide procedure, risultati documentati e ruoli definiti.

Gli obiettivi sono trovare i difetti, comprendere il prodotto e generare delle discussioni tra i membri, scambiare la conoscenza e decidere con coscienza.

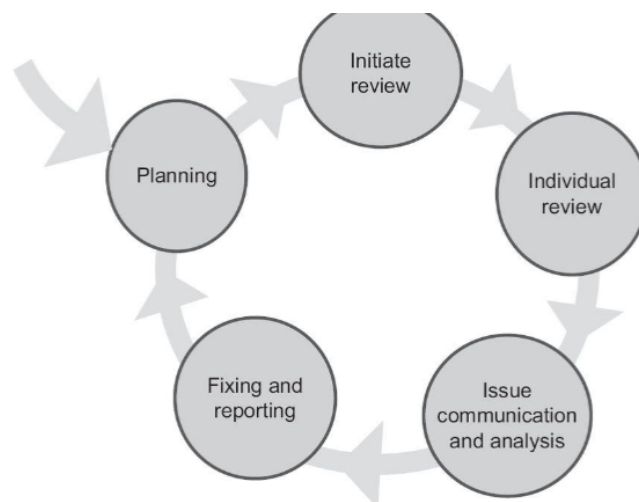


Figura 11.2: Schema di review.

Dove:

- **PLANNING:** definire lo scopo, stimare l'effort necessario, selezionare i partecipanti a cui assegnare gli oggetti da revisionare e i rispettivi ruoli.
- **INIZIALIZZAZIONE:** condividere documenti e artefatti, spiegare e chiarificare.
- **FASE INDIVIDUALE:** il revisionatore individua tutte le parti e cerca dei difetti.
- **PROBLEMI E ANALISI:** tramite meeting comunica gli errori tramite dei log.
- **FIXING:** gli autori correggono gli errori e verificano le metriche richieste.

## 11.5 Black Box Testing

**Black Box Testing:** attività di testing senza avere la conoscenza del funzionamento interno del software, come se fossimo un utilizzatore.

**Vantaggi:** posso mettermi in una prospettiva diversa ed esistono tanti tester senza skill specifiche che posso far lavorare anche senza conoscenze.

**Svantaggi:** la copertura del codice può essere limitata, può essere inefficiente se non conosco come funziona il codice e sono difficili.

1. **Equivalence class partitioning:** il comportamento dell'utente è riconoscibile da classi di equivalenza, in cui vengono raggruppati dei valori con cui il software si comporta nello stesso modo, ovvero che se quella classe ha un errore allora è previsto in tutte le classi di equivalenza. Definisco dei vincoli sugli attributi di tali classi e quando li testavo devo valutarle tutte almeno una volta, l'unica cosa proibita è testare più classi invalide.
2. **Boundary values analysis:** sfrutta il fatto che spesso gli errori sono dovuti ai confini delle classi di equivalenza non ad un elemento della classe. Anche qui è proibito testare più valori critici alla volta. Si testa di solito il valore minimo e appena sopra, massimo e appena sotto e uno nominale.
3. **Decision table:** adatte a contesti complessi permettono di rappresentare per ogni combinazione di condizioni di ingresso il valore delle uscite. Posso anche raggruppare con indifferenza alcune combinazioni, oppure posso in alternativa utilizzare i grafi causa effetto che sono equivalenti.
4. **Use-case testing:** permette di verificare scenari tipici e eccezionali, per garantire una copertura completa su tutti i flussi di lavoro, anche quelli meno percorsi. Per effettuare tali test si possono combinare altre tecniche viste e a volte è richiesto la conoscenza di informazioni aggiuntive.

## 11.6 White Box Testing

**White Box Testing:** attività di testing che valuta internamente il codice.

**Vantaggi:** posso apportare modifiche in diretta sul codice e ottimizzarlo.

**Svantaggi:** è difficile mantenere i test di questo tipo, servono dei tester competenti e richiedono tool speciali.

1. **Grafici controllo di flusso:** si tratta di creare rappresentazioni visive per il flusso del codice, definendo quanto e quali percorsi vanno testati.
  - (a) **Statement coverage:** tramite le regole di Beizer ci permette di verificare che tutte le linee del codice siano coperte.
  - (b) **Decision coverage:** permette di percorrere in entrambi i modi i nodi decisionali del codice, garantendo quindi anche la statement coverage.
  - (c) **Condition coverage:** analoga alla copertura decisionale se le condizioni sono atomiche, altrimenti le decompongo e le valuto vere e false.
  - (d) **Decision condition coverage:** si tratta dell'unione delle due precedenti tecniche per garantire la copertura completa del codice branch.

2. **Path Analysis:** si tratta di analizzare solamente i percorsi più corti possibili, evitando iterazioni, scegliendo solo quelli sensati e semplici. La teoria di **McCabe** dice che esistono alcuni percorsi base che vengono riutilizzati per garantire la copertura. È articolata nell'analisi del numero dei percorsi, inclusione o meno degli stessi nel testing, infine testing dei percorsi linearmente indipendenti che introducono un nuovo arco oltre i precedenti. Il numero di **percorsi logici** è  $2^{\text{numero di decisioni}}$ , mentre il numero di **percorsi linearmente indipendenti** è il numero cicломatico del grafo aumentato di 1.
3. **Testing di esperienza:** permette di indovinare gli errori, fare **testing esplorativo**, controllare una **check list** o utilizzare software come [SpotBugs](#).

## Capitolo 12

# Design Pattern

L'obiettivo di un design pattern è individuare gli aspetti della tua applicazione che variano e distinguerli da quelli che rimangono invariati. Questo è possibile programmando delle **interfacce** e non delle implementazioni. Esistono diversi tipi di design pattern:

| Scope |        | Purpose                                               |                                                                             |                                                                                                                      |
|-------|--------|-------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                            | Structural                                                                  | Behavioral                                                                                                           |
|       | Class  | Factory Method                                        | Adapter                                                                     | Interpreter<br>Template Method                                                                                       |
|       | Object | Abstract Factory<br>Builder<br>Prototype<br>Singleton | Adapter<br>Bridge<br>Composite<br>Decorator<br>Facade<br>Flyweight<br>Proxy | Chain of<br>Responsibility<br>Command<br>Iterator<br>Mediator<br>Memento<br>Observer<br>State<br>Strategy<br>Visitor |

Figura 12.1: Classificazione dei design pattern.

## 12.1 Strategy Pattern

Definisci una famiglia di algoritmi, incapsula ciascuno di essi e rendili intercambiabili. Strategy consente all'algoritmo di variare indipendentemente dai client che lo utilizzano.

### Applicabilità:

- Molte classi correlate differiscono solo nel loro comportamento.
- Hai bisogno di diverse varianti di un algoritmo.
- Un algoritmo utilizza dati che i clienti non dovrebbero conoscere.
- Una classe definisce molti comportamenti, che si manifestano come molteplici istruzioni condizionali nelle sue operazioni.

### Conseguenze:

- Famiglie di algoritmi collegati.
- Alternativa a sottoclassi.
- Rimozione degli statement condizionali.
- Overhead sulla comunicazione fra Strategy e Context.
- Numero elevato di oggetti.

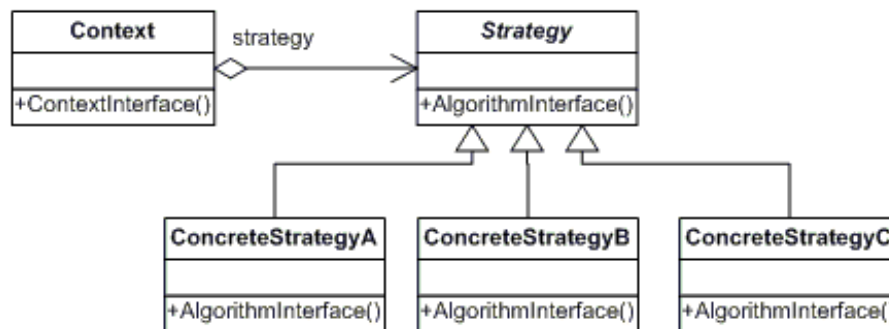


Figura 12.2: Struttura del design pattern Strategy.

Dove:

- **Strategy:** dichiara un'interfaccia comune a tutti gli algoritmi supportati. Il **context** utilizza questa interfaccia per richiamare l'algoritmo definito da una **ConcreteStrategy**.
- **ConcreteStrategy:** implementa l'algoritmo utilizzando l'interfaccia Strategy.
- **Context:** È configurato con un oggetto ConcreteStrategy. Mantiene un riferimento a un oggetto Strategy. Può definire un'interfaccia che consente a Strategy di accedere ai suoi dati.

## 12.2 Observer Pattern

Si tratta di un pattern che tiene informati i tuoi oggetti quando accade qualcosa che potrebbe interessarli. Utilizza un meccanismo **Publisher-Subscriber**. Un'entità pubblica informazioni, un'altra entità, che si iscrive, riceve automaticamente le informazioni. Nel momento che non vuole più riceverne, può disiscriversi.

### Applicabilità:

- Quando un'astrazione ha due aspetti, uno dipendente dall'altro.
- Quando una modifica a un oggetto richiede la modifica di altri oggetti e non si sa quanti oggetti debbano essere modificati.
- Quando un oggetto dovrebbe essere in grado di notificare altri oggetti senza fare supposizioni sull'identità di questi oggetti.

### Conseguenze:

- Accoppiamento astratto tra soggetto e osservatore.
- Supporto per le comunicazioni broadcast.
- Aggiornamenti imprevisti.

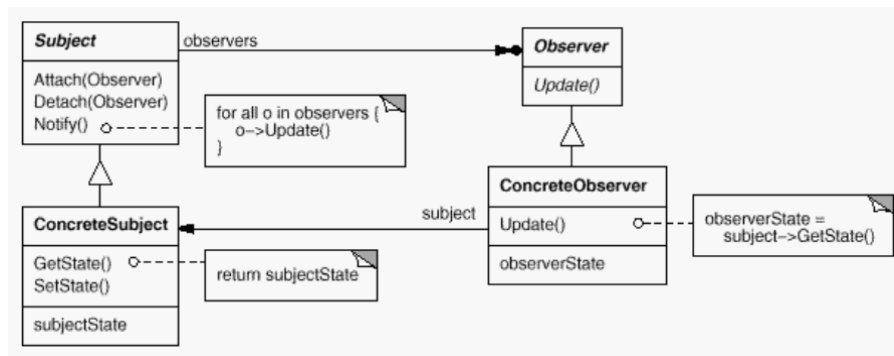


Figura 12.3: Struttura del design pattern Observer.

Dove:

- **Subject:** conosce tutti i suoi **observer**. Ogni observer può avere un soggetto da osservare, tramite le funzione di **detach** e **attach** di ogni soggetto.
- **Observer:** interfaccia di aggiornamento per gli oggetti che devono essere notificati delle modifiche a un soggetto.
- **ConcreteSubject:** memorizza lo stato di interesse degli oggetti **ConcreteObserver** e invia una notifica ai suoi osservatori quando il suo stato cambia.
- **ConcreteObserver:** implementa l'interfaccia **Observer**.

## 12.3 Decorator Pattern

I decoratori hanno lo stesso supertipo degli oggetti che decorano, possiamo passare un oggetto decorato al posto dell'oggetto originale. Il decoratore aggiunge il proprio comportamento prima e/o dopo aver delegato all'oggetto decorato il resto del lavoro. Gli oggetti possono essere decorati in qualsiasi momento, quindi possiamo decorarli dinamicamente in fase di esecuzione con tutti i decoratori che desideriamo.

### Applicabilità:

- Per aggiungere responsabilità ai singoli oggetti in modo dinamico e trasparente, ovvero senza influenzare altri oggetti.
- Per responsabilità che possono essere revocate.
- Quando l'estensione tramite sottoclassi non è praticabile.

### Conseguenze:

- Maggiore flessibilità rispetto all'ereditarietà statica.
- Evita classi ricche di funzionalità situate in alto nella gerarchia.
- Tanti piccoli oggetti.

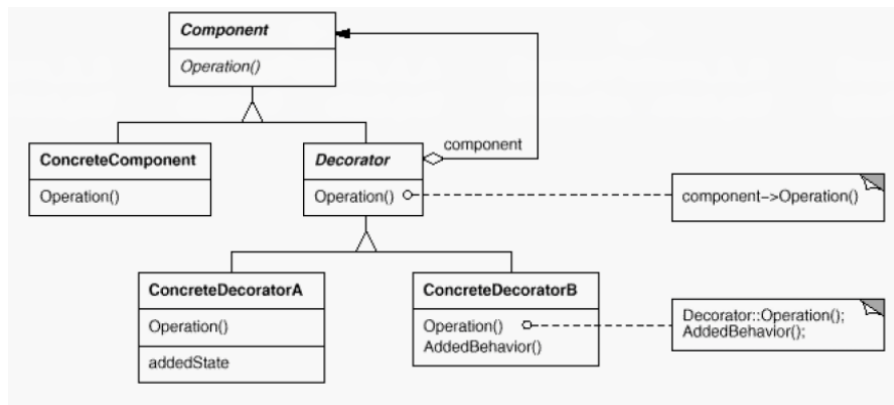


Figura 12.4: Struttura del design pattern Decorator.

Dove:

- **Component:** definisce l'interfaccia per gli oggetti a cui possono essere aggiunte responsabilità in modo dinamico.
- **ConcreteComponent:** definisce un oggetto al quale possono essere associate responsabilità aggiuntive.
- **Decorator:** mantiene un riferimento a un oggetto **Component** e definisce un'interfaccia che si conforma all'interfaccia del **Component**.
- **ConcreteDecorator:** aggiunge responsabilità al componente.

## 12.4 Factory Pattern

## 12.5 Singleton Pattern

Garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale ad essa.

### DEFINITION Continuity

A function  $f$  is continuous at  $a$  if  $\lim_{x \rightarrow a} f(x) = f(a)$ .

### WORKED EXAMPLE Prova

Prova.